

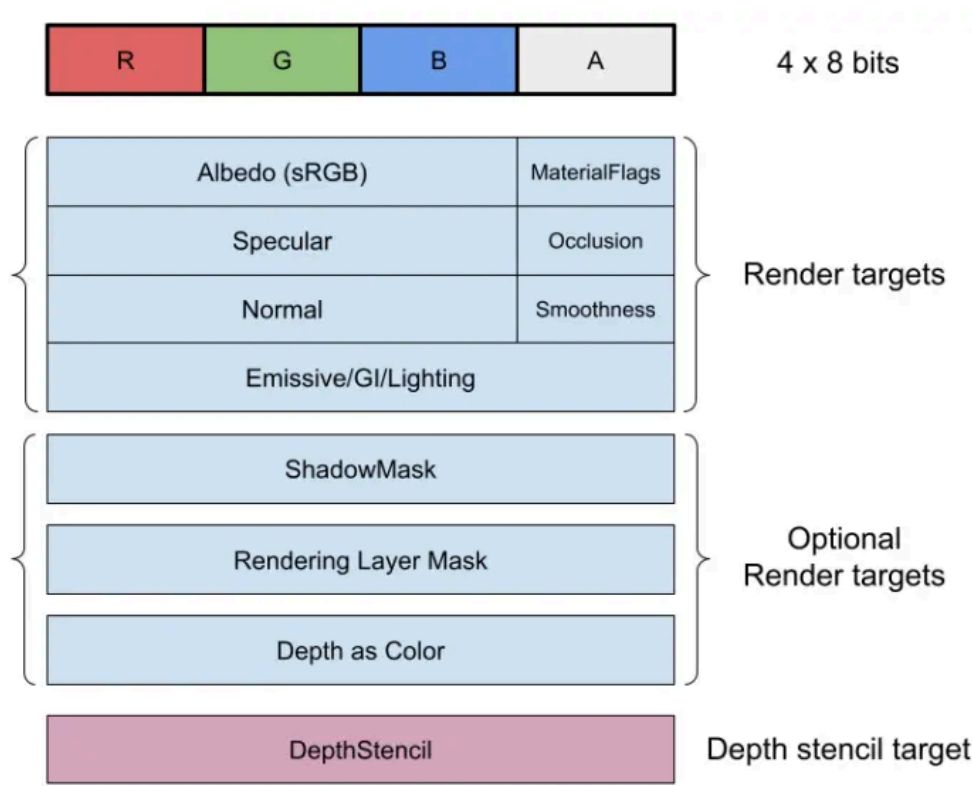
渲染管线

通用场景

概述

使用延迟渲染管线渲染几何物体，统一使用PBR材质。
然后使用向前渲染来渲染半透明物体。

Unity的G-buffer：



渲染管线-1767764207111.webp

我计划的GBuffer：

GBuffer 纹理	通道分配	格式
纹理 1	RGB=Albedo(sRGB); A=MaterialFlags	VK_FORMAT_R8G8B8A8_SRGB （适配 sRGB 颜色空间）
纹理 2	RGB=Specular; A=Occlusion	VK_FORMAT_R8G8B8A8_UNORM （无符号归一化，0~1 范围）
纹理 3	RGB=Normal; A=Smoothness	VK_FORMAT_R8G8B8A8_UNORM （Normal 需先压缩到 0~1 范围）
纹理 4	R= 着色 ID; GB = 自发光亮度; A = 自发光色相	VK_FORMAT_R8G8B8A8_UNORM

独立的深度附件

单独配置一个深度附件，无需放入 GBuffer：

附件类型	格式	说明
深度附件	VK_FORMAT_D32_SFLOAT	32 位浮点深度，精度足够覆盖绝大多数场景（若场景是小范围近距离，也可以用 VK_FORMAT_D16_UNORM）

延迟渲染管线

几何阶段

使用一个shader，分别渲染所有物体的G-buffer。

着色阶段

组装shader，使用分支语句把多个shader合并，然后根据 **着色ID**来渲染不同的shader。

默认shader设置为 **0**，也就是PBRshader。

参考PBRshader：

这里是硬分支，不太好，制作的时候要改成 glslc 编译器的 **模块化拆分 + 预处理合并**

TODO：待学习

```
// 采样你的GBuffer（重点是纹理4的解析）
layout(set = 0, binding = 1) uniform sampler2D gbuffer1; // Albedo + MaterialFlags
layout(set = 0, binding = 2) uniform sampler2D gbuffer2; // Specular + Occlusion
layout(set = 0, binding = 3) uniform sampler2D gbuffer3; // Normal + Smoothness
layout(set = 0, binding = 4) uniform sampler2D gbuffer4; // 着色ID + 自发光(亮度+色相)

// 光照数据（CPU端传入）
layout(set = 1, binding = 0) uniform LightData {
    vec3 lightDir;
    vec3 lightColor;
    vec3 viewDir;
} light;

// （可选）着色逻辑额外参数（按着色ID索引）
layout(set = 1, binding = 1) uniform ShaderExtraParams {
    float cartoonLineWidth; // 卡通着色描边宽度
    float pbrRoughnessScale; // PBR粗糙度缩放
} extra[256]; // 对应8位着色ID

// 自发光颜色计算（色相+亮度转RGB）
vec3 getEmissiveColor(float hue, vec2 brightness) {
    // 1. 色相（0-255）转角度（0-360°）
    float h = hue * 360.0 / 255.0;
    // 2. 亮度（GB通道）转0-65535范围（准HDR）
    float l = (brightness.r * 255.0) * 256.0 + (brightness.g * 255.0);
    l = l / 65535.0; // 归一化到0-1（也可保留大于1的值实现HDR自发光）
    // 3. HSL转RGB（色相h，饱和度s=1.0，亮度l）
    vec3 rgb = hslToRgb(vec3(h, 1.0, l)); // 需实现HSL转RGB函数（下方附）
    return rgb;
}
```

```

// 不同着色逻辑（沿用之前的分支，仅新增自发光计算）
vec3 shadePBR(vec3 albedo, vec3 normal, float smoothness, vec3 emissive) {
    float roughness = 1.0 - smoothness * extra[materialID].pbrRoughnessScale;
    vec3 specular = texture(gbuffer2, uv).rgb;
    float occlusion = texture(gbuffer2, uv).a;
    // PBR光照计算
    vec3 diffuse = max(dot(normal, light.lightDir), 0.0) * albedo * light.lightColor;
    vec3 halfDir = normalize(light.lightDir + light.viewDir);
    vec3 spec = pow(max(dot(normal, halfDir), 0.0), 64.0) * specular;
    //计算IBL
    //TODO
    return (diffuse + spec) * occlusion + emissive; // 叠加自发光
}

vec3 shadeCartoon(vec3 albedo, vec3 normal, vec3 emissive) {
    // 卡通漫反射（二值化）
    float diffuse = dot(normal, light.lightDir) > 0.6 ? 1.0 : 0.2;
    vec3 cartoonColor = albedo * diffuse * light.lightColor;
    // 描边
    float depthDiff = length(fwidth(texture(depthAttachment, uv).r));
    if (depthDiff > extra[materialID].cartoonLineWidth) cartoonColor = vec3(0.0);
    return cartoonColor + emissive; // 叠加自发光
}

void main() {
    vec2 uv = gl_FragCoord.xy / viewportSize;

    // 1. 解析GBuffer（重点：纹理4的新通道）
    vec4 g1 = texture(gbuffer1, uv);
    vec4 g2 = texture(gbuffer2, uv);
    vec4 g3 = texture(gbuffer3, uv);
    vec4 g4 = texture(gbuffer4, uv); // 关键：新通道分配

    vec3 albedo = g1.rgb;
    uint materialFlags = uint(g1.a * 255.0);
    vec3 specular = g2.rgb;
    float occlusion = g2.a;
    vec3 normal = normalize(g3.rgb * 2.0 - 1.0); // 还原法向量到[-1,1]
    float smoothness = g3.a;

    // 纹理4解析（核心调整）
    uint materialID = uint(g4.r * 255.0); // R通道=着色ID
    vec2 emissiveBrightness = g4.gb; // GB=自发光亮度（0-1范围）
    float emissiveHue = g4.a * 255.0; // A=自发光色相（0-255）
    vec3 emissive = getEmissiveColor(emissiveHue, emissiveBrightness); // 计算自发光颜色

    // 2. 按着色ID切换着色逻辑
    vec3 finalColor;
    switch(materialID) {
        case 0 ... 100: finalColor = shadePBR(albedo, normal, smoothness, emissive);
    }
    break;
}

```

```

        case 101 ... 200: finalColor = shadeCartoon(albedo, normal, emissive); break;
        case 201 ... 255: finalColor = emissive; break; // 纯自发光材质
        default: finalColor = shadePBR(albedo, normal, smoothness, emissive);
    }

    outColor = vec4(finalColor, 1.0);
}

// 辅助函数: HSL转RGB (适配自发光色相计算)
vec3 hslToRgb(vec3 hsl) {
    float h = hsl.x / 360.0;
    float s = hsl.y;
    float l = hsl.z;
    float c = (1.0 - abs(2.0 * l - 1.0)) * s;
    float x = c * (1.0 - abs(mod(h * 6.0, 2.0) - 1.0));
    float m = l - c / 2.0;
    vec3 rgb;
    if (h < 1.0/6.0) rgb = vec3(c, x, 0.0);
    else if (h < 2.0/6.0) rgb = vec3(x, c, 0.0);
    else if (h < 3.0/6.0) rgb = vec3(0.0, c, x);
    else if (h < 4.0/6.0) rgb = vec3(0.0, x, c);
    else if (h < 5.0/6.0) rgb = vec3(x, 0.0, c);
    else rgb = vec3(c, 0.0, x);
    return rgb + m;
}

```

向前渲染管线

主要用于添加半透明物体渲染。

独有的材质，使用透明shader。

混合模式**Alpha 混合**：需要启用颜色混合

（VkPipelineColorBlendStateCreateInfo 中 logicOpEnable = VK_FALSE，且混合因子非 VK_BLEND_FACTOR_ONE / VK_BLEND_FACTOR_ZERO），

- 正常混合：srcColorBlendFactor = VK_BLEND_FACTOR_SRC_ALPHA，dstColorBlendFactor = VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA （srca * 前景 + (1-srca) * 背景）；

TODO:

- 光线步进
 - 体积云
 - 体积雾
 - SDF几何体

后处理管线

- SSAO
 没时间就直接只做SSAO
 有精力的话把HBAO的论文读了，顺便直接继承SSR
- Bloom
 简单卷积一下算了

降采样多级卷积

- Tonemapping
 - 实现伽马调节，HDR映射到SDR
- Additive Fog
距离雾、高度雾

TODO：高斯点云场景

后面闲来无事再做。

处理高斯点云，解析每个点云的

- 位置
- 方向
- 缩放
- RGBA颜色
制作高斯点云shader
- 重点需要解决高斯点云混合的问题
 - 较为困难，分块、映射、重叠判断、视锥剔除等等